Knowledge Distillation and Cloud-Native Deployment of a Lightweight Command-Parsing Language Model

Course: CS3081 Cloud Computing | **Student:** Sabavat Sachith Naik (Roll: 230101093)
GitHub Repository: https://github.com/Nani779-hub/230101093-cloud-miniproject

Executive Summary: This project implements an end-to-end, resource-constrained sequence-level knowledge distillation pipeline and production-grade cloud microservice architecture. A high-parameter teacher model (Phi-3.5-mini-instruct, 3.8B parameters) was leveraged to generate semantic parse representations (intent, date, time, and place) for 3,000 commands from the SLURP benchmark. Teacher outputs were parallel-ingested via sharded Kubernetes workers into SeaweedFS S3-compatible object storage, cleaned, and verified against strict accounting identities. The resulting dataset was utilized to fine-tune an EleutherAI/Pythia-70M student model on pure CPU hardware. The student model was subsequently containerized into a minimal (<1.65 GB) FastAPI service, deployed on a multi-replica local k3d Kubernetes cluster with HPA autoscaling, zero-downtime rolling updates, health probes, and stateless Network Access Control Lists (NACLs), rigorously validated under CI/CD, and benchmarked under Locust load testing.

1. Task 1: Object Storage and Parallel Teacher Labelling

Architecture & Storage Setup: A local lightweight Kubernetes cluster was deployed via cluster/k3d.yaml with host-forwarded ports for HTTP/HTTPS ingress (8080) and S3 object storage (8333). Inside the data namespace, SeaweedFS was provisioned providing S3-compliant object storage with buckets datasets and models. The unlabelled pool of 3,000 natural language commands was uploaded to s3://datasets/commands_pool.jsonl. The teacher model (Phi-3.5-mini, 3.8B parameters) was served locally via Ollama with fixed hyperparameters (temperature=0, seed=42). A system prompt (prompts/teacher_prompt.txt) strictly constrained the output to a four-key JSON frame: intent, date, time, and place across the eight SLURP intents (alarm_set, alarm_query, calendar_set, calendar_query, calendar_remove, weather_query, datetime_query, recommendation_events).

Parallel Sharding Strategy: To distribute workload without inter-worker coordination or locking, deterministic modulus sharding was implemented: worker *k* out of *N* processes record *i* if i mod N == k. Two independent Kubernetes Jobs (teacher-worker-0 and teacher-worker-1) ran concurrently in the pipeline namespace, each processing 1,500 commands and checkpointing outputs to s3://datasets/teacher/shard-0.jsonl and shard-1.jsonl every 10 records. Shard accounting verified exactly 1,500 unique commands per shard with zero overlap and zero gaps across the 3,000 commands.

Data Cleaning & Quality Gate (clean.py): Shards were merged and sanitized against criteria (a) through (e). The strict mathematical identity input = (a) + (b) + (c) + (d) + duplicates + kept (3000 = 1032 + 0 + 0 + 46 + 34 + 1888) was verified:

Category / Reason	Count	Percentage	Description / Action
Total Input Records	3,000	100.0%	Combined lines across shard-0 and shard-1
(a) Invalid JSON	1,032	34.4%	Model returned malformed syntax, truncated braces, or non-JSON
(b) Wrong Keys	0	0.0%	Missing required keys or containing extraneous keys
(c) Invalid Intent	0	0.0%	Intent outside the eight sanctioned assignment intents
(d) Hallucinated Slots	46	1.5%	Slot text not verbatim present in natural language input command
(e) Duplicate Commands	34	1.1%	Redundant utterances; retained entry with smallest command ID
Valid Clean Records Kept	**1,888**	**62.9%**	**Preserved in ID order; saved to datasets/v1/train.jsonl**

SHA-256 Checksum: Cleaned dataset was uploaded to s3://datasets/v1/train.jsonl with hash a6ba785ca8cc3ab5dfe853cd01371178ce265f383279cf176ba99463656ea6ad, saved in datasets/v1/train.sha256 and data/train.sha256 with Unix LF formatting.

2. Task 2: Training the Student (Sequence-Level Distillation)

Motivation & Distillation Theory: Deploying modern LLMs (>3B parameters) on low-power edge nodes induces prohibitive memory footprints and latency bottlenecks. Sequence-Level Knowledge Distillation (Kim & Rush, EMNLP 2016) mitigates this by training a compact student model (Pythia-70M, 70M parameters) directly on teacher-generated natural language sequences: Command: <text>\nJSON: <teacher_reply>. In contrast to token-level or logit-based distillation (which minimizes Kullback-Leibler divergence over full teacher softmax distributions P_T(y_t | y_{<t}, x)), sequence-level distillation requires only the teacher's greedily decoded surface outputs y* = argmax_y P_T(y|x).

Why Logit-Based Distillation is Infeasible Here: (1) The teacher model (Phi-3.5, vocabulary size 32,064) and student model (Pythia-70M, vocabulary size 50,304) possess non-aligned token vocabularies and differing subword tokenizers (TikToken BPE vs. GPT-NeoX BPE), making direct per-token cross-entropy alignment mathematically non-trivial. (2) Standard inference runtimes (such as Ollama REST API) expose generated text tokens rather than raw logit distribution tensors across all 32,000+ dimensions. (3) Sequence-level cross-entropy loss on teacher outputs avoids the massive memory transfer overhead of saving full probability distributions.

Implementation & Hyperparameters: As specified in training/config.yaml, training was executed on CPU using PyTorch and Hugging Face Transformers with seed 42, AdamW optimizer (learning rate 5.0e-5, weight decay 0.01), batch size 8, sequence length 80, linear warmup, and gradient clipping at 1.0. Measured epoch loss from training/loss.csv confirmed monotonic convergence: Version 1 (5 epochs on 1,500 records) decreased from 2.2158 to 0.2915 (2.2158 -> 0.8992 -> 0.5757 -> 0.4162 -> 0.2915); Version 2 converged to 0.1716 at epoch 7 (... -> 0.2213 -> 0.1716). Both versions were uploaded to SeaweedFS under models/v1/ and models/v2/.

Genuine Empirical Evaluation on validation.jsonl (100 Commands):

Model	Parameters	JSON Validity	Intent Acc.	Date Acc.	Time Acc.	Place Acc.	Full Frame
Teacher (Phi-3.5-mini)	3.8 B	70.0%	62.0%	59.0%	62.0%	53.0%	**33.0%**
Untrained Student (Pythia-70M)	70 M	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Trained Student v1 (5 epochs)	70 M	**99.0%**	**76.0%**	46.0%	79.0%	82.0%	**27.0%**
Trained Student v2 (7 epochs)	70 M	94.0%	72.0%	60.0%	79.0%	83.0%	**40.0%**

Analysis & Limitations: Untrained Pythia-70M produces 0.0% valid JSON, as it lacks domain knowledge of structured syntax. Post-distillation, Student v1 JSON validity reaches **99.0%** (passing the >= 95% quality gate) with **27.0%** full-frame accuracy (>= 16.5% assignment floor). Student v2 achieves **40.0%** full-frame accuracy via improved date extraction (60.0% vs 46.0%), but drops to **94.0%** JSON validity, failing the quality gate. Hence, v1 is selected for production.

3. Task 3: The Inference Service in a Container

Service Architecture: Implemented in service/app.py with FastAPI and Uvicorn. Exposes three core endpoints: (1) POST /parse parsing natural language commands into 4-key JSON with response headers X-Model-Version and X-Served-By; (2) GET /healthz returning HTTP 200 for liveness; (3) GET /readyz returning HTTP 503 during model download and HTTP 200 once weights are fully loaded into memory.

Externalized Weight Retrieval & Secret Ingestion: Image layers strictly exclude model weights. On startup, the service queries S3 using Boto3 configured with environment variables (MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS) and credentials populated exclusively from a Kubernetes Secret (s3-credentials).

Schema Validation & Fault Injection: Generated outputs are validated against cloud_assignment/data/schema.json. Malformed JSON or schema deviations trigger HTTP 500 with {"error": "invalid_model_output"}. With FAULT_INJECT=1, model output is overridden with "not json", validating fail-safe error handling.

Container Optimization & 50-Request Benchmark: Built via multi-stage Dockerfile utilizing CPU-only PyTorch (torch==2.6.0+cpu), stripping CUDA/cuDNN binaries. Final image disk footprint is **1.64 GB** (under the 2.0 GB ceiling). Running 50 sequential requests under cgroup limits of 1 vCPU and 1 GiB memory yielded a measured peak memory of **333.9 MiB (0.326 GiB)** with zero OOM events. 48 requests returned HTTP 200; 2 requests returned HTTP 500 because raw model outputs contained syntax errors caught by the schema validator, demonstrating active error protection.

4. Task 4: Kubernetes Deployment, Scaling, and Reliability

Cluster Topology & Ingress Routing: Deployed to namespace app on k3d Kubernetes. Ingress routing maps port 8080 traffic to the ClusterIP Service on port 80, forwarding to pod port 8000. Traefik handles Layer 7 routing and round-robin load distribution.

Horizontal Pod Autoscaling (HPA): Configured with min 1, max 2 replicas, target CPU utilization 70%. Under load, metrics-server registered high CPU, automatically triggering scaling from 1 to 2 replicas, and scaling back down post-cooldown.

Zero-Downtime Rolling Updates & Rollback: During active traffic, deployment configuration was updated from v1 to v2 using maxUnavailable: 0, maxSurge: 1, and a container preStop: ["/bin/sleep", "3"] hook to allow Ingress endpoint deregistration before process exit. Under continuous Locust load, 119 requests were served across versions (50 on v1, 69 on v2) with zero connection drops (HTTP 502/503 = 0) and zero HTTP 500 failures (0.00% error rate, meeting the <= 0.5% threshold). A subsequent kubectl rollout undo deployment/command-parser -n app successfully restored X-Model-Version: v1 with 2 Ready replicas.

Pod Recovery & Cold Start: Deleting an active pod during load testing resulted in immediate replica replacement, restoring 2 Ready pods in under 20 seconds with 0 connection drops. Cold-start benchmarks (measuring duration from pod creation to readyz HTTP 200 across 3 clean runs) recorded **72.58s, 9.82s, and 19.02s** (median: **19.02s**).

Load Balancing Distribution: Locust header analysis across 2 healthy replicas over 2 minutes showed Replica A served **51.0% (75 requests)** and Replica B served **49.0% (72 requests)**, comfortably satisfying the 30% balance rule.

5. Task 5: Subnetting and Network Access Control (NACL)

Subnet Partitioning (10.20.0.0/16): Allocated consecutively from largest host requirement to smallest using minimal power-of-two prefixes 2^(32-prefix) - 2 >= Hosts:

Subnet	Hosts Req.	Network CIDR	Prefix	Usable Host Range	Broadcast	Usable Hosts
app	1,000	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

Stateless NACL Theory & Rule Ordering: Stateless NACLs do not preserve session state; each inbound and outbound packet is evaluated independently. Return web traffic requires explicit outbound ephemeral port rules (1024–65535). Rules are evaluated in strict ascending order: rule

100 explicitly blocks abusive range 198.51.100.0/24 across all ports before rule 110/120 allows broad 0.0.0.0/0 traffic. Verified 6/6 (100%) via cloud_assignment/tools/nacl_sim.py.

6. Task 6: Reproducibility, Testing, and CI/CD

One-Command Lifecycle: make up creates the k3d cluster, provisions SeaweedFS, builds Docker images, and deploys services. make down safely tears down all cluster resources.

GitHub Actions Pipeline (.github/workflows/ci.yml): Implemented with 4 sequential stages: (1) lint (flake8, 0 errors/warnings); (2) test (28 automated pytest unit tests across 4 test suites covering cleaning, service schema, pipeline, and NACL rules, all passing); (3) evaluate (configured to download v1.0.0 from GitHub Release assets, verify SHA256 checksums, generate validation predictions, and enforce score.py --min-validity 95); (4) secret-scan (Gitleaks automated scan configured with if: always() && !cancelled(), verified locally with 0 leaks).

Quality Gate Failure Verification: Quality gate failure logic was empirically verified using untrained Pythia-70M predictions (results/untrained_predictions.jsonl), where score.py correctly exited with code 1 (json_validity: 0.0% < 95.0%). The CI workflow includes a test_bad_model: true dispatch parameter and a failing-gate-demo branch trigger to demonstrate the genuine quality-gate failure remotely upon release asset upload.

7. Task 7: Performance, Cost, and Comprehensive Review

Locust Load Test Measurements (2 minutes, 4 users): Total requests: **151**, successful HTTP 200: **147**, throughput: **1.28 req/s**, median latency (p50): **2,500 ms**, 95th percentile latency (p95): **6,300 ms**. Pod resource utilization during load (kubectl top pods): CPU was **554m and 549m**; memory was **349 MiB and 362 MiB** (peak RAM: **362 MiB**). On this laptop hardware (Intel i5-1334U, CPU-only execution inside 1-core cgroups), 4 unthrottled concurrent users cause queueing delays that push p95 to 6,300 ms, and the 1 validation syntax anomaly (V042) cycling repeatedly yields a 2.65% HTTP 500 rate.

Cost Estimation (rate_card.json): Hourly cost per replica = (1 vCPU * 3.50 INR) + (1 GiB * 0.45 INR) = **3.95 INR/hr**. For 2 running replicas: **7.90 INR/hr**. Requests served per hour = 147 req / 120s * 3600s = **4,410 req/hr**. Cost per 1,000 requests = (7.90 / 4,410) * 1,000 = **1.79 INR** (or **2.39 INR** under conservative 3,300 req/hr baseline).

SLO Definition & Error Budget: Target: **95.0%** HTTP 200 responses over 2-minute load window (151 total requests). Allowed error budget: **7.55 requests**. Actual errors observed: **4 requests** (53.0% of budget consumed, 47.0% remaining). Measured success rate: **97.35%**.

Critical Review Table (6 Dimensions):

Component / Concern	Evidence (Measured Value)	Limitation Identified	Proposed Improvement
Data & Storage	1,888 kept records; 34 duplicates removed; 19.02s median cold start	S3 object retrieval on pod startup introduces ~19s init delay	Pre-warm models via local node SSD DaemonSet caching
Model Quality	99.0% JSON validity; 27.0% full frame vs 33.0% teacher	Teacher labeling errors are propagated and memorized	Multi-teacher ensemble distillation with majority voting
Latency & Throughput	p50: 2,500 ms; p95: 6,300 ms; throughput: 1.28 req/s	CPU token autoregression limits concurrency scaling	Quantize to INT8/INT4 (ONNX/vLLM) to double throughput
Resource Utilisation	362 MiB peak RAM; 554m peak CPU under 1 core / 1 GiB limit	Static 1 GiB limit causes headroom underutilization	Tune limits to 512 MiB with VPA memory recommendation
Reliability & Recovery	0 dropped connections on rolling update; <20s pod recovery	Single-node laptop cluster lacks multi-host fault tolerance	Deploy multi-zone topology with pod anti-affinity rules
Security & Cost	1.79–2.39 INR / 1k req; 0 gitleaks detected; 6/6 NACL passed	Stateless NACLs require broad ephemeral return port ranges	Combine NACLs with stateful Cilium / Calico NetworkPolicies

Key References: (1) Kim & Rush, 'Sequence-Level Knowledge Distillation', EMNLP 2016; (2) Bastianelli et al., 'SLURP: A Spoken Language Understanding Resource Package', EMNLP 2020; (3) Biderman et al., 'Pythia: A Suite for Analyzing Large Language Models', ICML 2023; (4) Google SRE Book, 'Service Level Objectives', O'Reilly Media; (5) Kubernetes Networking & Ingress Architecture Guide.